ZentroMart: AI Customer Support Agent

Capstone Project Report

Abstract

This project develops an educational AI customer-support assistant for ZentroMart, a fictional Indian e-commerce company. The prototype retrieves relevant sections from company knowledge documents, classifies common support intents, performs identity-gated synthetic order lookups, searches a product catalogue and creates human-handover tickets. It prioritizes traceability and privacy by showing source section IDs and withholding order details until verification succeeds.

1. Problem Statement

Customers frequently ask repetitive questions about order tracking, returns, refunds, cancellations, payments and product issues. A support assistant must find the applicable current policy, avoid superseded guidance, protect order-specific information and escalate cases that require human judgment.

2. Objectives

(1) Build a searchable knowledge base from Markdown documents. (2) Retrieve evidence for natural-language questions. (3) Detect common support intents. (4) Protect order information using identity verification. (5) Support human escalation through ticket creation. (6) Evaluate retrieval against supplied practice questions.

3. Dataset

The synthetic dataset includes 300 customers, 1,000 orders, 1,783 order-item rows, 70 products, 36 pincode records, 4 holidays, 244 support logs, 21 knowledge documents, 52 practice questions and 23 multi-turn practice conversations. The fixed dataset reference date is 20 October 2026. No real customer records are used.

4. Methodology

Knowledge documents are parsed into heading-based chunks. Each chunk retains its section ID, filename, document type, status and effective date. Superseded documents are excluded. TF-IDF word bigrams and character n-grams represent chunks and queries. Cosine similarity is combined with 0.65 weight for word features and 0.35 for character features. The highest-ranked evidence chunks are returned with source IDs.

5. System Architecture

Customer message → intent detection → knowledge retrieval → ranked policy/help evidence → grounded response with source IDs. For order-specific requests, the agent additionally uses the verification-gated order tool. Cases requiring human review are passed to the ticket-creation tool.

6. Implementation

Python provides application logic and orchestration. Pandas handles catalogue data. Scikit-learn provides TF-IDF vectorization and cosine similarity. Streamlit provides the interactive interface. The supplied local tools provide verified order lookup and ticket creation.

7. Evaluation

Run python evaluate.py to retrieve the top five evidence sections for every question. The first local run retrieved at least one expected gold evidence section in the top five for 33 of 47 questions with populated gold-evidence labels: hit@5 = 70.2%. Five questions without a gold evidence label were excluded. This is a retrieval metric, not an answer-accuracy score. Detailed results are in evaluation_results.csv.

8. Results and Discussion

The prototype provides a reproducible retrieval baseline and visible source traceability. Word features capture direct terminology, while character n-grams provide limited robustness to spelling variation. Keyword intent detection is explainable but is not a trained classifier. The extractive answer strategy reduces unsupported generation but may be less natural than a properly grounded LLM.

9. Limitations

This is not a production deployment. TF-IDF is not semantic transformer retrieval; Hindi/Hinglish performance is limited. The mock identity check is educational only. No live LLM or voice provider is configured because API credentials were not supplied. The synthetic data must not be treated as real customer data.

10. Conclusion

ZentroMart demonstrates the core components of a retrieval-grounded customer-support agent: knowledge ingestion, evidence retrieval, intent routing, identity-gated order access and human handover. Future work can add multilingual sentence embeddings, a vector database, a carefully constrained LLM, automated policy tests and the optional voice interface.

How to Run

1. Install dependencies: pip install -r requirements.txt

2. Validate data: python validate_dataset.py

3. Start interface: streamlit run app.py

4. Run retrieval evaluation: python evaluate.py

Project files: app.py, agent.py, rag_engine.py, evaluate.py, data/, kb/, practice/, tools/.